Desktop Search Engine using Inverted Index

NLP Lab Assignment Report

Abdul-Rahman Tahir

BSDSF23A038

1. Introduction

With the rapid growth of digital data, efficient information retrieval systems have become essential. Search engines allow users to quickly locate relevant documents from large collections of text. The purpose of this lab assignment was to design and implement a desktop search engine capable of indexing and retrieving documents from a large dataset of local files.

The system was implemented using an **inverted index structure**, which is widely used in modern information retrieval systems. The implemented search engine supports multiple vector representations including **Boolean vectors, Count vectors, and TF–IDF vectors**. Additionally, two similarity measures, **Inner Product and Cosine Similarity**, were implemented to rank documents according to their relevance to a query.

The search engine was designed to handle **50,000 text files**, demonstrating scalability and efficient retrieval performance.

---

2. Dataset

For this project, a dataset consisting of **50,000 text documents** was used. Since collecting such a large number of real documents can be difficult, the dataset was programmatically generated using a Python script. Each document contains text related to topics such as:

- Machine learning
- Natural language processing
- Cricket and sports
- Databases and SQL
- Cloud computing
- Cybersecurity
- Data science

This approach allowed the system to simulate a realistic text corpus while ensuring that the indexing and retrieval system could handle a large number of documents.

Each file was stored in `.txt` format inside a dataset directory. During indexing, each document was assigned a unique **document ID (docID)** and mapped to its file path for retrieval.

---

3. Preprocessing Pipeline

Before building the index, all documents were processed through a preprocessing pipeline. The goal of preprocessing is to convert raw text into a clean and standardized form suitable for indexing.

The following preprocessing steps were implemented:

3.1 Tokenization

Tokenization divides the text into individual words or tokens. This allows the system to analyze the frequency and presence of each term within the documents.

Example:

```
"Machine learning is powerful"
```

becomes

```
["machine", "learning", "is", "powerful"]
```

3.2 Lowercasing

All tokens were converted to lowercase to ensure case-insensitive searching. Without this step, words such as *Machine* and *machine* would be treated as different terms.

3.3 Punctuation Removal

Special characters and punctuation marks were removed to avoid unnecessary tokens. This ensures that only meaningful words remain in the dataset.

---

4. Inverted Index Construction

The core component of the system is the **inverted index**. An inverted index maps each term to a list of documents in which that term appears.

Instead of scanning every document for every query, the inverted index allows the system to quickly retrieve the relevant documents.

The index structure follows this format:

```
term → [(docID, term frequency)]
```

Example:

```
"cricket" → [(5,2), (10,1), (24,3)]
```

This means:

- Document 5 contains the word *cricket* 2 times
- Document 10 contains it once
- Document 24 contains it 3 times

During indexing, the system also stored:

- **Document frequency (df)** for each term
- **Term frequency (tf)** within each document
- **Document vector norms** for cosine similarity
- **Document ID to file path mapping**

The inverted index was saved to disk using Python serialization so that it can be loaded later without rebuilding the index.

---

5. Vector Representations

To represent documents and queries numerically, three different vector models were implemented.

5.1 Boolean Vector Model

In the Boolean model, a document vector contains only binary values:

```
1 → term appears in the document
0 → term does not appear
```

This model simply identifies whether a term exists in a document but does not consider how frequently it appears. Although simple, this model is useful for basic keyword searching.

5.2 Count / Frequency Vector

In this model, the weight of each term is equal to its **term frequency (TF)** in the document.

Formula:

```
w(t,d) = tf(t,d)
```

Documents that contain a term multiple times receive higher weights. This approach provides better ranking than the Boolean model because it reflects the importance of frequently occurring terms.

5.3 TF–IDF Vector Model

The TF–IDF model improves ranking by considering both **term frequency** and **inverse document frequency**.

The formulas used were:

```
idf(t) = log((N + 1) / (df(t) + 1)) + 1
w(t,d) = tf(t,d) × idf(t)
```

Where:

- **N** = total number of documents
- **df(t)** = number of documents containing term *t*

This method reduces the influence of very common words and gives more importance to terms that are unique to certain documents.

---

6. Similarity Measures

To determine how relevant a document is to a query, similarity measures were implemented.

6.1 Inner Product (Dot Product)

The inner product computes similarity by multiplying corresponding weights of query and document vectors.

```
score(d,q) = Σ w(t,d) × w(t,q)
```

Documents with higher scores are considered more relevant.

6.2 Cosine Similarity

Cosine similarity measures the angle between two vectors. It normalizes the scores by dividing by the magnitude of the vectors.

```
score(d,q) = dot(d,q) / (||d|| × ||q||)
```

Cosine similarity produces scores between **0 and 1**, making it a more reliable measure for comparing document relevance.

---

7. Query Processing

When a user enters a query, the system performs the following steps:

1. The query is preprocessed using the same pipeline as the documents.
2. Tokens are matched against the inverted index.
3. Document scores are calculated using the selected vector model and similarity measure.
4. The documents are ranked based on their scores.
5. The **Top-k results (default k = 10)** are displayed.

Each result includes:

- Rank
- File path
- Similarity score

---

8. Bonus Features

Two additional features were implemented to improve search capabilities.

Phrase Queries

Phrase queries return documents containing an exact phrase.

Example:

```
"machine learning"
```

The system searches for documents where the words appear in the same order.

Proximity Queries

Proximity queries allow searching for terms that appear within a certain distance.

Example:

```
pakistan cricket ^5
```

This query returns documents where the words *pakistan* and *cricket* appear within five words of each other.

---

9. Conclusion

In this lab assignment, a desktop search engine capable of indexing and retrieving information from a large document collection was successfully implemented. The system supports multiple vector representations and similarity measures, demonstrating the core principles of information retrieval systems.

The use of an inverted index significantly improved search efficiency, allowing the system to handle large datasets efficiently. The implementation of TF–IDF and cosine similarity further improved the accuracy of document ranking.

Overall, this project provided practical experience in designing scalable text retrieval systems and applying key concepts from natural language processing and information retrieval.

10. Experiments and Comparisons

The experimental results demonstrate that the choice of vector representation and similarity measure has a significant impact on search quality. While the Boolean model provides

fast retrieval, it lacks ranking capability. The count model improves this by incorporating term frequency, but still does not account for term importance across documents.

TF–IDF weighting significantly improves ranking by reducing the impact of common terms. When combined with cosine similarity, the search engine achieves the best performance by accurately measuring the similarity between queries and documents.

These findings highlight why **TF–IDF and cosine similarity are widely used in modern information retrieval systems**.

Boolean

```
Enter query (or 'exit'): cricket match
Choose mode (boolean / count / tfidf / cosine): boolean

Rank | File Path
1 dataset\doc_0.txt
2 dataset\doc_10.txt
3 dataset\doc_100.txt
4 dataset\doc_1000.txt
5 dataset\doc_10000.txt
6 dataset\doc_10001.txt
7 dataset\doc_10002.txt
8 dataset\doc_10003.txt
9 dataset\doc_10005.txt
10 dataset\doc_10006.txt
```

Count

```
Enter query (or 'exit'): cricket match
Choose mode (boolean / count / tfidf / cosine): count

Rank | File Path | Score
1 dataset\doc_10913.txt 18
2 dataset\doc_40999.txt 18
3 dataset\doc_48584.txt 18
4 dataset\doc_49983.txt 18
5 dataset\doc_13975.txt 16
6 dataset\doc_14127.txt 16
7 dataset\doc_21270.txt 16
8 dataset\doc_25940.txt 16
9 dataset\doc_27122.txt 16
10 dataset\doc_28034.txt 16
```

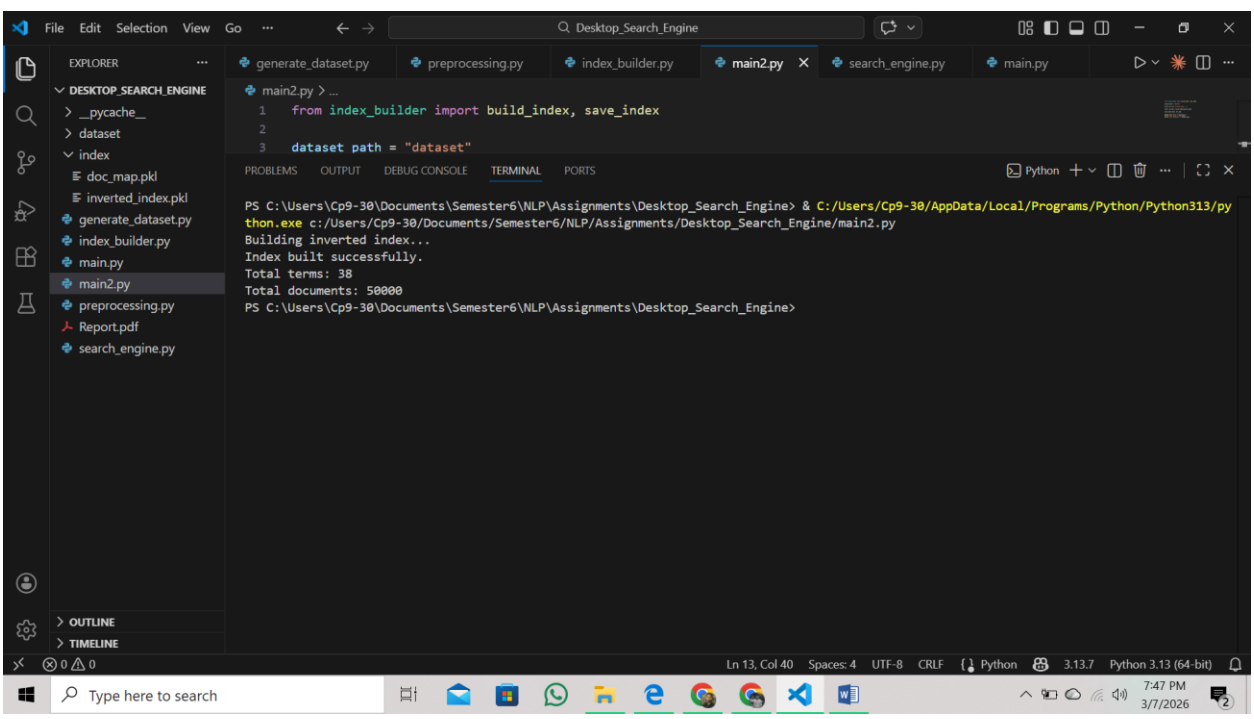
Tf-idf

```
Enter query (or 'exit'): cricket match
Choose mode (boolean / count / tfidf / cosine): tfidf

Rank | File Path | Score
1 dataset\doc_10913.txt 20.33247847486827
2 dataset\doc_40999.txt 20.33247847486827
3 dataset\doc_48584.txt 20.33247847486827
4 dataset\doc_49983.txt 20.33247847486827
5 dataset\doc_13975.txt 18.07331419988291
6 dataset\doc_14127.txt 18.07331419988291
7 dataset\doc_21270.txt 18.07331419988291
8 dataset\doc_25940.txt 18.07331419988291
9 dataset\doc_27122.txt 18.07331419988291
10 dataset\doc_28034.txt 18.07331419988291
```

Cosine

```
Enter query (or 'exit'): cricket match
Choose mode (boolean / count / tfidf / cosine): cosine

Rank | File Path | Score
1 dataset\doc_49983.txt 0.05444231044589156
2 dataset\doc_10913.txt 0.0543868257242652
3 dataset\doc_40999.txt 0.05330867015255692
4 dataset\doc_28034.txt 0.05251677134973367
5 dataset\doc_48584.txt 0.05247669868863228
6 dataset\doc_47566.txt 0.052113418237228905
7 dataset\doc_31681.txt 0.052103082384886514
8 dataset\doc_14127.txt 0.05132470690035007
9 dataset\doc_5284.txt 0.05112062256838196
10 dataset\doc_27122.txt 0.05111511062878471
```

Index building output